AI PIXEL ART — COMPLETE PROJECT BRIEF

A complete reference for building an AI-assisted pipeline that produces high-quality pixel art sprites from 3D models, with aesthetic evaluation and per-pixel editing tooling.

PART ONE — PROJECT THESIS (READ THIS FIRST)

The Core Insight

You are building a tool that lets an AI draw pixel art the way an AI can draw — not by mimicking how a human pixel artist works.

The standard approach to AI image generation tries to make AI perceive and produce images the way humans do: look at pixels visually, judge them aesthetically, generate pixel-by-pixel output evaluated by human visual perception. This forces AI through a human-shaped sensory bottleneck and makes the AI slower, weaker, and more brittle than it needs to be.

This project does the opposite. The AI works in its native representation — structured data, palette grids, addressable cells, structural rules, reference deltas — and a deterministic renderer translates that data into PNG pixel art at the final step. The AI never tries to "see" pixels. The AI reasons about data; the renderer produces pixels.

This is the same pattern that already works for AI code generation. AI doesn't write code by typing keys at human speed on a visual editor. AI emits text directly into files. The interface is plain text — AI-native, not human-native — and as a result AI codes faster and more reliably than any approach that tries to make AI mimic a human typing.

We're applying that same principle to pixel art.

What You Are Actually Building

A pipeline with these properties:

1. Every stage emits data, not just pixels. Each stage's output must be inspectable as text or JSON. The AI participant in the loop reasons about data, not images.
2. The AI's working medium is structured representations: palette index grids, palette band

distributions, structural metrics, reference comparisons. NOT screenshots of rendered output.

3. The renderer is a deterministic translation layer at the END of the pipeline. It converts data to PNG. The AI does not perceive its output visually — the renderer is for human consumption.
4. The reference library is the bridge between mechanical correctness and aesthetic quality. The AI compares its output's structural metrics to reference distributions and corrects toward the reference target. This replaces "does this look good" (which AI cannot answer) with "does this match the structural pattern of references humans rated highly" (which AI can answer trivially).
5. Mechanical errors and aesthetic problems are separate. Mechanical errors (orphan pixels, broken outlines, band skips) are deterministic and fixed by code rules. Aesthetic problems (proportions, palette choice, "feels right") require reference comparison. Don't conflate them.

The Architectural Insight That Matters

When designing any stage of this pipeline, ask:

> "What is the AI-native representation of this task?"

NOT:

> "How would a human artist do this task?"

A human pixel artist looks at a sprite and intuitively judges the highlight placement. The AI cannot do this. So we don't ask the AI to do it. Instead, we encode the rule: "highlight cluster centroid should be in the upper-left quadrant of the silhouette bounding box, and 2-8% of body area." That rule is data. The AI can verify it, enforce it, and correct toward it.

A human pixel artist intuitively knows when transitions look jarring. The AI cannot perceive jarring. So we encode: "no two adjacent pixels should differ by more than one palette band." Data. Verifiable. Correctable.

Every aesthetic insight from real pixel art needs to be translated into this kind of structural rule. The reference library is how we capture the aesthetic insights we can't articulate as rules — by storing examples and comparing structural distributions.

The Anti-Pattern To Avoid

Do not build features where the AI tries to "look at" rendered output and make decisions.

Examples of the anti-pattern:

- “AI inspects screenshot of bake and decides what to fix”
- “AI judges if the slime looks like a slime”
- “AI evaluates aesthetic quality by examining the PNG”

These all force the AI through human visual perception. Wrong layer.

Correct versions of those features:

- “AI inspects palette grid (as text/JSON) and applies cleanup rules”
- “AI compares slime’s structural metrics against reference distribution and flags deviations”
- “AI computes FID against labeled reference set”

Same goal. AI-native interface. Faster, more reliable, deterministic.

Translation Layers Are Cheap

It’s tempting to skip the data-first approach because “we have to render the PNG eventually anyway, why not just work with the PNG?” The answer:

The translation layer (data → PNG) is cheap. It’s a deterministic renderer. Maybe 200 lines of code. It runs in milliseconds.

The reverse direction (PNG → understanding what’s in the PNG) is expensive. That’s where the entire field of computer vision lives. Trying to make the AI work backwards from PNG to understanding is what makes AI tools brittle.

So: do the work in data space. Render to PNG only at the boundary where a human needs to see the result. Never round-trip through PNG mid-pipeline.

What Success Looks Like

You’ll know the architecture is right when:

- The AI can iterate on the pipeline by reading text dumps of pixel grids and modifying generation rules — without ever seeing a rendered image
- Adding a new aesthetic constraint means encoding a structural rule or adding labeled references, not training a vision model
- The same data flows through the pipeline whether you’re rendering a slime, a wisp, a

humanoid, or a building

- Quality improvements come from better data structures and better reference comparison, not from "AI looking harder at pixels"
- The system is debuggable: any pipeline failure can be traced to a specific stage's data output, inspectable as text

What This Project Is Not

This is not "AI image generation." This is not "AI that draws pixel art." This is not a Stable Diffusion alternative.

This is a deterministic pipeline where AI authors data structures and a renderer produces pixels. The AI's contribution is reasoning about structure, applying rules, and matching references — all in data space.

If your implementation finds itself reaching for image generation models, vision models, or any "AI looks at images" component, you've drifted off the architecture. Come back to: what data does the AI need, what rules does it apply, what references does it compare against, and what does the renderer produce as output.

PART TWO — TECHNICAL BRIEF

Section 1 — Rendering Pipeline Methods

Method A — Native Low-Resolution Rendering

WHAT: Render the 3D scene directly at the target sprite resolution (e.g. 64x64), with anti-aliasing disabled. No downsampling involved.

WHY: Each output pixel corresponds to one shader invocation. No averaging means no fuzzy edges, no quantization noise from multiple samples collapsing into one pixel.

WORKS: Yes. This is the correct approach used by t3ssel8r, BUKKBEEK, David Holland, Roystan, and most production pipelines.

DOES NOT WORK: Rendering at high res (256x256) and downsampling to 64x64. Produces soft, fuzzy edges because nearest-neighbor downsampling picks one pixel from a 4x4 block; depending on which block-pixel lands on an edge, the silhouette flickers between frames.

Method B — Cel/Toon Shading with Discrete Bands

WHAT: Replace continuous Lambert shading with N discrete brightness bands. Each pixel gets one of `{shadow_deep, shadow, mid, bright, highlight, peak}` based on `dot(normal, lightDir)` thresholds.

WHY: Pixel art is defined by flat color regions, not gradients. PBR shaders produce gradients that look like noise at low resolution.

FORMULA:

```
intensity = max(0, dot(normal, lightDirection))
if      (intensity < 0.18) col = shadow_deep
else if (intensity < 0.40) col = shadow
else if (intensity < 0.62) col = mid
else if (intensity < 0.84) col = bright
else                       col = highlight
```

TYPICAL: 3-5 bands. More than 5 starts to look like banded gradient. Less than 3 looks flat.

Method C — Five Lighting Components for Form

WHAT: Layer five separate lighting calculations to build full form:

1. Multi-band Lambert (multi-band shadow) — main form
2. Specular highlight (Blinn-Phong, hard-stepped) — shimmer
3. Rim light (fresnel, hard-stepped) — translucence/gel feel
4. Ambient occlusion (vertex-down or SSAO baked) — depth
5. Bayer 2x2 dither at band boundaries — soft transitions

FORMULAS:

```
// Specular (single hard-stepped band)
H = normalize(L + V)
spec = pow(max(0, dot(N, H)), 32) * specStrength
specBand = step(0.65, spec)

// Rim (fresnel)
rim = 1.0 - max(0, dot(N, V))
rim = pow(rim, 2.5)
rimBand = step(0.55, rim) * rimStrength

// Vertex-down AO
```

```
ao = 1 - aoStrength * smoothstep(0, -1, N.y) * 0.5

// Bayer 2x2 dither
bayer2(p) = floor(p) -> idx 0,1,2,3 maps to 0, 0.5, 0.75, 0.25
intensity += (bayer2(fragCoord) - 0.5) * ditherAmt * (1.0 / nBands)

// Compose: brightest of [intensity, rim, spec]
final = max(intensity, max(rimBand, specBand))
```

Method D — Hue-Shifted Palette

WHAT: Palette colors don't just brighten/darken — they shift hue. Shadows shift cool (purple/blue). Highlights shift warm (yellow).

WHY: Real pixel art (Hollow Knight, Octopath, Vampire Survivors, Castlevania) has hue-shifted palettes. Pure brightness ramps look mechanical.

EXAMPLE PALETTE (8 colors, crimson aesthetic):

```
transparent  rgba(0,0,0,0)
outline      #1c0814   (purple-shifted dark, NOT pure black)
shadow_deep  #380c20   (purple-tinted)
shadow       #681424   (red-purple)
mid          #b8281c   (saturated red, dominant body)
bright       #e86024   (warm orange-red)
highlight    #ffa840   (warm orange-gold)
peak         #ffe088   (pale warm yellow, NOT pure white)
```

Method E — Depth-Based Outline Pass

WHAT: Render scene depth to a separate buffer. Apply Roberts Cross edge detection. Pixels at depth discontinuities get drawn in outline color over the main image.

FORMULA:

```
For each pixel (x, y):
  d_self = sampleDepth(x, y)
  if d_self == background: skip
  d_neighbors = [d(x, y-1), d(x, y+1), d(x-1, y), d(x+1, y)]
  if any d_neighbor == background: this pixel is silhouette edge
  maxDiff = max(|d_self - d_n| for n in neighbors)
  if maxDiff > 0.10 (normalized): this pixel is interior crease
  if either: paint with outline color
```

WORKS: Roberts Cross is fast and produces 1-pixel-thick outlines. Sobel/Canny work but

are heavier.

ALTERNATIVE: Inverted hull mesh (flipped normals, scaled outward, rendered black). Cheaper but doesn't catch interior creases.

Method F — Roberts Cross Edge Detection (post-process)

WHAT: 2x2 convolution kernel applied to depth and normal buffers.

KERNEL:

```
Gx = [[1, 0],      Gy = [[0, 1],
      [0, -1]]           [-1, 0]]
```

USAGE: Apply at internal render resolution (NOT full screen). This guarantees 1-pixel outline thickness regardless of upscale.

Method G — Texel-Matching Camera (David Holland / t3ssel8r)

WHAT: Snap camera position to pixel grid. Compensate UVs with sub-pixel fraction so geometry shows continuous motion despite camera snapping.

WHY: Otherwise sub-pixel camera motion creates pixel jitter as the sprite moves. With snapping + UV compensation, motion is smooth and pixels stay stable.

FORMULA:

```
snappedCamPos = floor(camPos * pixelsPerUnit) / pixelsPerUnit
fractional = (camPos - snappedCamPos) * pixelsPerUnit
uvOffset = fractional / renderTextureSize
sample texture with (UV + uvOffset)
```

Section 2 — Post-Bake Cleanup Passes (Data Transformations)

These run AFTER the GPU produces pixels. They operate on the pixel grid as a 2D array of palette indices. Each is a deterministic rule.

Pass 1 — Orphan Removal

RULE: Pixel whose color appears in NONE of its 4 neighbors AND is rare globally (<5px of that color) → replace with most common neighbor color. **WHY:** Quantization edge cases produce single stray pixels.

Pass 2 — Outline Thinning

RULE: If outline pixel has 3+ outline neighbors AND has a body neighbor → demote to shadow color. **RULE:** 2x2 outline blocks → demote bottom-right to shadow. **WHY:** Real pixel art outlines are exactly 1px thick.

Pass 3 — Outline Gap Repair

RULE: Body pixel adjacent to transparent pixel → convert to outline. **WHY:** Edge detector occasionally misses pixels just below threshold.

Pass 4 — Highlight Cluster Validation

RULE: Flood-fill to find peak-color clusters. Keep largest only. Demote others to highlight. **WHY:** Shader noise can scatter peak highlight into multiple specks.

Pass 5 — Single-Pixel Limb Removal

RULE: Body pixel with 3+ transparent neighbors → set transparent. **WHY:** 1-pixel protrusions are almost always rendering artifacts.

Pass 6 — Band Skip Prevention

RULE: If two adjacent pixels are 3+ palette levels apart, replace one with the intermediate level. **WHY:** Real pixel art transitions through bands sequentially.

Pass 7 — Highlight Area Enforcement

RULE: Count peak pixels. If >10% of body, demote outer-ring peak pixels to highlight. **WHY:** Peak highlight should be 2-8% of body area in shipped sprites.

Pass 8 — Mid-tone Dominance (diagnostic)

RULE: Count mid-color ratio. If <30% or >70%, flag the bake as structurally unbalanced. **WHY:** Mid tone should dominate body (~40-60%) per reference analysis.

Pass 9 — Bottom Contact Shadow

RULE: Find lowest body pixel per column. Darken bottom 2 pixels by one palette band. **WHY:** Ground-resting creatures need contact shadow to look grounded.

Section 3 — Animation Frame Counts (Research-Backed)

NEVER use uniform-time-sampling for an animation. Use keyframes with explicit per-frame durations. The “snap” between poses is what makes attacks feel weighty; smooth interpolation makes them feel underwater.

Frame counts per animation type, by era:

Animation	NES	SNES	Modern Indie	Modern AAA-2D
Idle	1-2	2-4	2-4	4-8
Walk/Run	2-3	4-6	4-6	6-10
Attack	2-4	4-6	4-8	8-15
Jump	3-4	5-7	4-9	8-12

Per-frame timing (the secret):

For a 6-frame attack at total ~900ms:

```
Frame 1 (idle/neutral)    100ms
Frame 2 (wind-up mid)     130ms
Frame 3 (wind-up max)     180ms   <- held longer to telegraph
Frame 4 (strike/impact)   240ms   <- HELD LONGEST (the snap)
Frame 5 (recover early)    90ms
Frame 6 (settle)          160ms
```

The impact pause (200-300ms held strike frame) is the Hollow Knight nail-swing principle. Without it, the attack feels limp.

Reference data points:

- Celeste's Madeline: 4-frame run cycle
- Shovel Knight: 6-frame walk cycle
- Stardew Valley: 4-frame walk
- Dead Cells: 8-12 frame attacks (large sprites, fluid combat)
- Hollow Knight Hornet (NOT pixel art, hand-drawn): ~400 frames TOTAL across all of her animations, breaks down to ~6-12 frames per individual move

Sub-pixel animation (Metal Slug technique):

For idle animations specifically — keep silhouette mostly stable, let interior pixels shift colors. The "breathing" comes from internal shading variation, not silhouette displacement. This is achieved by:

- Body scale variation of only 1-2% during idle

- Larger variation in lighting band thresholds (so internal shading shifts more than silhouette)

Section 4 — Aesthetic Evaluation Methods

The hard problem: how do you score "this looks like good pixel art"?

Method A — FID (Fréchet Inception Distance)

WHAT: Statistical distance between feature distributions of generated images and a reference set. Lower = closer to reference.

HOW: Run both reference set and generated batch through Inception-v3 penultimate layer. Compute mean and covariance of feature vectors.

```
FID = ||μ_ref - μ_gen||² + Tr(Σ_ref + Σ_gen - 2*sqrt(Σ_ref*Σ_gen))
```

WORKS WELL FOR: large reference sets (100+ images), bulk evaluation. **DOESN'T:** single-image feedback, tiny reference sets.

Method B — Discriminator Network (GAN style)

WHAT: Train a CNN to classify images as "real pixel art" vs "fake/poor". Trained on labeled pairs.

HOW: Standard GAN discriminator architecture. Patch-based training (2x2 patches work best per Serpa & Rodrigues research — better than 5x5, 11x11, 64x64 single patch).

WORKS: Once trained, gives a real-time score on any candidate image. **NEEDS:** Training data with quality labels (~1000+ examples minimum).

Method C — Structural Comparison

WHAT: Encode pixel art "rules" as measurable properties. Compare generated output to reference distribution on each axis.

MEASURABLE PROPERTIES:

- Palette band distribution ratios (% mid, % shadow, % highlight)
- Highlight cluster size (% of body) and centroid position
- Outline thickness consistency (variance across silhouette)
- Symmetry score (left-right pixel match for symmetric creatures)

- Color count (unique colors in image)
- Hue range (max-min hue across palette)
- Gradient sharpness (avg brightness delta at band boundaries)
- Silhouette aspect ratio (bounding box w/h)
- Single-color cluster sizes (largest cluster of each color)

COMPARE: For each property, compute mean and variance across reference set. Generated image gets score per property = z-score from reference distribution. Aggregate score = sum or weighted sum.

Method D — Pretrained Pixel Art Network

WHAT: Use existing trained models (SIGGRAPH Asia 2022 "Make Your Own Sprites: Aliasing-Aware and Cell-Controllable Pixelization").

HOW: Pass generated output through encoder-decoder pretrained on real pixel art datasets. Compute reconstruction error or activation distance from reference.

DATASETS USED IN LITERATURE:

- Multi-cell dataset (10K+ pixel art sprites, various games)
- Aliasing dataset (paired aliased/anti-aliased examples)

Section 5 — Reference Library Design

Storage format (per reference sprite):

```
{
  "id": "ref_001",
  "source": "shipped_game_X",
  "quality": "excellent",
  "tags": ["slime", "creature", "fantasy"],
  "resolution": [64, 64],
  "palette_size": 8,
  "grid": "<2D array of palette indices>",
  "metrics": {
    "highlight_ratio": 0.045,
    "shadow_ratio": 0.18,
    "mid_ratio": 0.52,
    "outline_thickness_variance": 0.0,
    "symmetry_score": 0.91,
```

```
    "unique_colors": 8,
    "silhouette_bbox": [12, 18, 50, 48]
  }
}
```

Ingestion pipeline:

1. Load image (PNG, transparent background required)
2. Auto-detect palette (cluster unique colors)
3. Quantize to N colors using k-means in LAB color space
4. Compute all structural metrics
5. Human assigns quality label
6. Store as JSON entry

Comparison query:

Given a generated sprite:

1. Compute its metrics
2. Find nearest references by metric distance
3. If nearest is “excellent” tier → output is on target
4. If nearest is “poor” tier → flag specific diverging metrics
5. Output: targeted feedback (“highlight_ratio 0.012, references average 0.045 — increase highlight cluster size”)

Recommended reference set size:

- Minimum viable: 30-50 labeled sprites
- Good: 100-200
- Production-grade: 500+

Section 6 — Technology Stack Options

Option 1 — Browser/JavaScript (Three.js)

PROS: Zero install, runs anywhere, hot reload, easy to share. WebGL gives shader access. ImageData provides pixel access. **CONS:** Limited to WebGL 1/2 (no compute shaders

without WebGPU). Heavier ML libraries (tensorflow.js) slower than Python. **USE FOR:** Prototyping, web tools, browser-based editors, demo artifacts. **LIBRARIES:**

- three.js — 3D rendering
- canvas2d — pixel manipulation, ImageData
- tensorflow.js — neural network inference (optional)

Option 2 — Python (PyTorch + ModernGL/PyOpenGL)

PROS: Full ML ecosystem (PyTorch, transformers, diffusers). Easy access to FID, discriminator training, dataset tools. Standard for academic pixel-art research. **CONS:** Slower iteration than browser. Requires environment setup. **USE FOR:** Training discriminators, computing FID, batch processing, neural network research. **LIBRARIES:**

- torch — neural network framework
- torchvision — image datasets, transforms
- pillow / PIL — image I/O
- numpy / scipy — array math
- moderngl — modern OpenGL bindings
- pyrender — offscreen 3D rendering
- scikit-image — image processing
- cleanfid — FID computation
- open_clip_torch — CLIP for semantic comparison

Option 3 — Godot Engine (GDScript / GDExtension)

PROS: Mature 3D-to-pixel pipeline already exists. Free, open source, lightweight, exports to web/desktop/mobile. Built-in SubViewport for render-to-texture. **CONS:** GDScript is Godot-specific. Less ML ecosystem. **USE FOR:** Final asset pipeline. Game runtime.

Option 4 — Blender (Python API)

PROS: Best 3D modeling tools available. Eevee renderer for fast pixel-style output. Blender 4.2+ has pixelate compositor node. **CONS:** Heavy. Better for offline batch processing than realtime. **USE FOR:** Asset creation, offline batch baking.

RECOMMENDED HYBRID:

- Blender or Three.js: 3D model authoring (procedural or hand)
- Godot: rendering pipeline (use existing tool's shaders)
- Python: aesthetic evaluation, training, batch comparison
- Browser: interactive tools, live preview, per-pixel editor

Section 7 — What Works / What Doesn't

WORKS:

- ✓ Native low-res rendering (NOT downsample from high-res)
- ✓ Cel/toon shading with 3-5 discrete bands
- ✓ Hue-shifted palettes (cool shadows, warm highlights)
- ✓ Roberts Cross outline pass at internal resolution
- ✓ Hard-stepped specular and rim (NOT smooth)
- ✓ Bayer 2x2 dither at band boundaries
- ✓ Tinted outlines (NOT pure black)
- ✓ Per-frame variable timing (NOT uniform sampling)
- ✓ Held impact frames (200-300ms strike pose)
- ✓ Procedural animation via phase state machines
- ✓ Slime/blob/wisp/multi-bodied creatures
- ✓ Cube-anatomy (Minecraft style — "cubist" creatures)
- ✓ Hierarchical skeletons + rotation animation
- ✓ FID-based evaluation against reference set
- ✓ Patch-based discriminator training (2x2 patches optimal)
- ✓ Sub-pixel animation for idle (silhouette stable, interior shifts)

DOESN'T WORK:

- × Anti-aliasing of any kind (browsers, MSAA, FXAA, TAA — all off)
- × PBR/realistic shading (gradients become noise at low res)

- × Pure black outlines (look harsh, don't integrate)
- × Pure white peak highlights (looks blown out)
- × Pure brightness ramp palettes (no hue shift = mechanical look)
- × High-poly geometry at low res (band boundaries become jaggy)
- × Uniform per-frame timing (animations feel mechanical)
- × Too many frames (>12 for attacks tends to look like 3D in slow-mo)
- × Gel/FBM/noise shaders for pixel art (gradients defeat the look)
- × AI generating humanoid hands/faces from primitives (too hard)
- × AI judging aesthetic quality without reference dataset
- × Single-pixel protrusions (always rendering artifacts)
- × 2-pixel-thick outlines (real pixel art is exactly 1px)

AI-SPECIFIC LIMITATIONS DISCOVERED:

- LLMs cannot perceive rendered pixel output. Must operate via:
  - ASCII grid representations
  - Structural metrics
  - Reference comparison
- 3D spatial reasoning with multiple transformations is unreliable. Especially: rotation + scale combinations that change which axis is "depth" vs "height" — silent bug source.
- Procedural humanoid anatomy (hands, faces, walking gaits) fails at high consistency. Procedural amorphous creatures (slimes, wisps, tentacle creatures) succeed at high consistency.
- "More rules" doesn't fix aesthetic problems. Rules catch mechanical errors. Aesthetic requires reference-based judgment.

AI-SPECIFIC STRENGTHS DISCOVERED:

- Phase state machines for combat animation
- Procedural shaders (FBM noise, fresnel, caustic, gel effects)
- Hierarchical bone systems with cascading rotations

- Animation playback systems, sprite sheet renderers
- System architecture, asset loading, animation blending
- Encoded mechanical cleanup rules
- Math/IK/easing curve implementation

Section 8 — Recommended Pipeline Architecture

A six-stage pipeline. Each stage exposes data; each stage is independently testable.

Stage 1 — Authoring

- **INPUT:** Designer intent
- **OUTPUT:** 3D model (low-poly, ~500-2000 tris) + animation data
- **TOOLS:** Blender (preferred), Three.js procedural (for amorphous shapes)
- **DATA:** GLB/glTF file + animation keyframes (rotation per bone per keyframe, with explicit duration per keyframe)

Stage 2 — Rendering

- **INPUT:** 3D model + animation + lighting setup
- **OUTPUT:** Raw color + depth buffers at native low resolution (e.g. 64x64)
- **TOOLS:** Three.js (browser) or Godot (desktop) or Blender Eevee (batch)
- **SHADERS:** Cel-shaded toon material (5 bands), depth-write material, orthographic camera, no anti-aliasing
- **DATA:** ImageData buffer (color), Float32Array (depth)

Stage 3 — Outline + Quantization

- **INPUT:** Color + depth buffers
- **OUTPUT:** 64x64 grid of palette indices
- **PROCESS:**
  1. Roberts Cross on depth → outline mask
  2. snapToPalette() on color → palette indices
  3. Composite outline over color

- **DATA:** Int8Array of length 64*64 (one palette index per pixel)

Stage 4 — Cleanup

- **INPUT:** Palette index grid
- **OUTPUT:** Cleaned palette index grid
- **PROCESS:** Apply passes 1-9 in order (orphan removal, outline thinning, outline gap repair, highlight cluster validation, single-pixel limb removal, band skip prevention, highlight area enforcement, mid-tone diagnostic, bottom contact shadow)

Stage 5 — Aesthetic Evaluation

- **INPUT:** Cleaned grid
- **OUTPUT:** Quality scores + flagged issues
- **PROCESS:**
  1. Compute structural metrics (palette ratios, highlight pos, outline thickness, symmetry)
  2. Compare to reference distribution (z-scores per metric)
  3. (Optional) Run FID against reference set
  4. (Optional) Run discriminator network
  5. Generate flag list: which metrics deviate from reference
- **DATA:** JSON `{metrics, scores, flags}`

Stage 6 — Export

- **INPUT:** Validated grid + animation metadata
- **OUTPUT:** PNG sprite sheet + JSON metadata
- **PROCESS:**
  1. Composite all frames into horizontal strip
  2. Generate metadata: frame durations, hitboxes, anchor points
- **DATA:** PNG file, JSON sidecar

Section 9 — ASCII Grid Representation (Perception Loop)

The critical infrastructure that lets an LLM “see” the output by reading text. Every stage’s output should be dump-able as ASCII.

Character mapping:

```
.  transparent
#  outline
X  shadow_deep
x  shadow
o  mid (main body)
O  bright
*  highlight
@  peak
```

Example 32x32 dump:

```
................................
................................
................#...............
...........#####O#####..........
.........##OOOOOooooo##........
........#OOooooooooooooo#.......
.......#OooOOOOOooooooooo#......
......#OoOOOO*OOOOxooooooo#.....
.....#OooOO**@**OOxxxxxoooo#....
.....#OoOOO*@@@*OOOXXxxxooo#....
.....#OooOO**@**OOXXXXXxxoo#....
....#OoooOOOO*OOOOXXXXXxxooo#...
.....#oooooOOOOOXXXXXXXxxxo#....
.....#ooooooxxXXXXXXXXXxxoo#....
```

Use cases:

- Debug rendering: did the bake produce expected output?
- Identify specific issues: “row 12 has band-skip from o to X”
- Compare frames: diff between frame N and frame N+1
- Reference comparison: paste reference as ASCII alongside generated

Implementation (Node.js example):

```
function gridToAscii(grid, palette) {
```

```
  const charMap = {
    transparent: '.', outline: '#',
    shadow_deep: 'X', shadow: 'x',
    mid: 'o', bright: 'O',
    highlight: '*', peak: '@',
  };
  return grid.map(row =>
    row.map(idx => charMap[palette[idx].name]).join('')
  ).join('\n');
}
```

Section 10 — Repository Structure

```
ai-pixel-art/
├── README.md
├── package.json                # if JS-based
├── pyproject.toml              # if Python-based
│
├── src/
│   ├── core/
│   │   ├── palette.js          # palette definitions, quantization
│   │   ├── grid.js             # 2D grid data structure + ops
│   │   └── ascii.js            # ASCII rendering for perception loop
│   │
│   ├── rendering/
│   │   ├── shaders/
│   │   │   ├── toon.frag       # cel-shaded fragment shader
│   │   │   ├── depth.frag      # depth-encoding shader
│   │   │   ├── outline.frag    # Roberts Cross edge detect
│   │   │   └── common.glsl     # shared GLSL utilities
│   │   ├── camera.js           # texel-matching orthographic camera
│   │   ├── lighting.js         # 5-component lighting setup
│   │   └── bake.js             # render-to-grid orchestration
│   │
│   ├── cleanup/
│   │   ├── pass1_orphan.js
│   │   ├── pass2_outline_thin.js
│   │   ├── pass3_outline_repair.js
│   │   ├── pass4_highlight_cluster.js
│   │   ├── pass5_single_pixel.js
│   │   ├── pass6_band_skip.js
│   │   ├── pass7_highlight_area.js
│   │   ├── pass8_mid_diagnostic.js
│   │   ├── pass9_contact_shadow.js
│   │   └── index.js            # pipeline runner
```

```
│   │
│   ├── eval/
│   │   ├── metrics.js          # structural measurements
│   │   ├── reference_lib.js    # load + query reference set
│   │   ├── compare.js          # generated vs reference scoring
│   │   ├── fid.py              # FID computation (Python)
│   │   └── discriminator.py    # GAN discriminator (Python)
│   │
│   ├── animation/
│   │   ├── keyframe.js         # keyframe data structure
│   │   ├── timing.js           # per-frame duration logic
│   │   ├── phase_machine.js    # phase state machines for combat
│   │   └── playback.js         # runtime sprite playback
│   │
│   ├── authoring/
│   │   ├── procedural/
│   │   │   ├── slime.js        # procedural slime (works well)
│   │   │   ├── wisp.js         # procedural wisp
│   │   │   ├── tentacle.js     # CatmullRom tube tentacle
│   │   │   └── blob.js         # procedural amorphous shapes
│   │   └── loader.js           # load external GLB/glTF
│   │
│   └── export/
│       ├── spritesheet.js      # composite frames to PNG strip
│       └── metadata.js         # JSON sidecar generation
│
├── references/
│   ├── images/                 # raw reference PNGs
│   ├── grids/                  # processed grid JSONs
│   └── manifest.json           # quality labels, tags
│
├── tests/
│   ├── unit/                   # individual function tests
│   └── golden/                 # golden ASCII outputs to diff against
│
├── tools/
│   ├── ascii_dump.js           # CLI: render any frame to ASCII
│   ├── reference_ingest.js     # CLI: add new reference sprite
│   ├── eval_run.js             # CLI: run evaluation on a bake
│   └── live_editor/            # browser-based pixel editor
│
└── docs/
    ├── pipeline.md             # this brief
    ├── shaders.md              # shader documentation
    ├── palette_design.md       # how to design palettes
    └── reference_curation.md   # how to label references
```

Section 11 — Implementation Phases

Phase 1 — Foundation (Week 1-2)

- Set up repo structure
- Implement palette + grid data structures
- Implement ASCII rendering
- Build basic Three.js scene with cel shader
- Verify perception loop: bake → ASCII → read

Phase 2 — Core Pipeline (Week 3-4)

- Implement all 9 cleanup passes
- Implement Roberts Cross outline
- Implement palette quantization
- Build CLI tools for bake + dump

Phase 3 — Animation (Week 5-6)

- Phase state machine framework
- Keyframe + per-frame timing
- Sprite sheet export (PNG + JSON)
- Playback runtime

Phase 4 — Reference Library (Week 7-8)

- Reference ingestion pipeline
- Structural metrics computation
- Label ~50 initial references manually
- Comparison + flagging system

Phase 5 — Evaluation (Week 9-10)

- FID implementation in Python
- (Optional) Train discriminator on reference set

- Integrate evaluation into pipeline

Phase 6 — Iteration Tooling (Week 11-12)

- Live editor (browser-based)
- Per-pixel editing operations
- Auto-iteration: detect flagged issues, propose targeted fixes
- Diff visualization (generated vs reference)

Section 12 — Key Design Principles

1. **EVERY STAGE EMITS DATA, NOT JUST PIXELS.** Each stage's output must be inspectable as text/JSON. The LLM participant in the loop reasons about data, not images.
2. **SEPARATE MECHANICAL FROM AESTHETIC.** Mechanical errors (orphans, broken outlines, band skips) are deterministic and fixed by code rules. Aesthetic problems (proportions, palette choice, "feels right") require reference comparison or human judgment. Don't conflate them.
3. **NATIVE RESOLUTION RENDERING.** Render at the final sprite resolution. Never downsample. This is the single biggest "what works / what doesn't" pivot.
4. **TIMING > FRAME COUNT.** Six well-timed frames beat twelve uniformly-sampled frames. Per-frame duration is the difference between snappy and limp.
5. **HUE SHIFT THE PALETTE.** Cool shadows, warm highlights. Pure brightness ramps look mechanical. This is non-negotiable for "shipped pixel art" feel.
6. **KEEP REFERENCES FIRST-CLASS.** The reference library isn't a nice-to-have. It's the bridge between mechanical rules and aesthetic judgment. Build it early.
7. **PROCEDURAL FOR AMORPHOUS, AUTHORED FOR ANATOMICAL.** Slimes, blobs, wisps, tentacle creatures: procedural works. Humanoids, hands, faces: use authored models from artists or asset stores, not procedural construction.
8. **THE LLM IS A CODE-AND-DATA REASONER.** It's good at: shaders, math, state machines, animation logic, data transformations, system architecture, reference encoding. It's bad at: predicting visual output, anatomical proportions, aesthetic judgment without references, perceiving its own work. Design accordingly.

PART THREE — COMPETITIVE BENCHMARK BRIEF

Section A — What PixelLab Actually Is

PixelLab is the current commercial leader in AI pixel art for game devs. Important: they do NOT publish FID scores, accuracy benchmarks, or any academic metrics. They compete on PRODUCT FEATURES and OUTPUT CONSISTENCY, not on numerical model quality.

PixelLab's stack:

- Two underlying models: "PixFlux" (medium-to-XL images) and "BitForge" (small-to-medium images). Closed-source, proprietary architectures.
- Cloud GPU inference (no local compute required for users).
- Browser web app + Aseprite plugin + REST API + MCP server.
- Built by a small team. Active Discord. Frequent model updates.

What PixelLab actually delivers (their feature surface):

- Text-to-pixel-art character generation
- 4 and 8 directional rotation views from a single concept image
- Reference-based generation (consistency across batches)
- Skeleton-based animation (walk, run, idle, attack)
- Text-to-animation
- Context-aware inpainting
- Wang tilesets for seamless map tiles
- AI upscaling that maintains pixel-art fidelity
- 4-direction sprite rotation in a single click
- Up to 400×400 pixel scenes (top tier)

Their pricing tiers (the resolution ceilings ARE the benchmark):

- Free trial: 200×200 max, 40 fast generations
- Apprentice ($12): 320×320 max, animation tools
- Artisan ($24): 400×400 max, priority queue, experimental

- Architect ($50): 400×400 max, 20 concurrent jobs, team tools

What competitors get wrong (per industry reviewers):

- Midjourney/Gemini NanoBanana: produce “mixels” (mixed pixel sizes where some pixels are 1×1 and others are 2×2 or 3×3, breaking the illusion of true pixel art)
- Most general AI tools: can’t handle transparency correctly
- Most general AI tools: inconsistent style across multiple generations
- Most general AI tools: produce only one specific style

What PixelLab does that competitors don’t:

- Pixel-perfect outputs (no mixels)
- Style consistency across generations of the same character
- True pixel-art transparency
- Game-ready format (sprite sheets, multi-direction, animation)

Section B — Academic Benchmarks That Exist

These are the real numbers you can actually target. Pixel art doesn’t have its own widely-adopted benchmark, so the field uses general image generation benchmarks that pixel art models can be tested against.

Benchmark 1 — FID (Fréchet Inception Distance)

THE STANDARD METRIC. Lower is better.

Reference scores from related work:

- PixelFlow (2024, pixel-space generative model): **1.98 FID** on 256×256 ImageNet class-conditional
- PixelFolder (2022, progressive pixel synthesis): **3.77 FID** on FFHQ (faces), **2.45 FID** on LSUN Church
- Sprite-flow (2024, pixel art specifically): No published score, but uses FID for evaluation in their codebase

Targeting: For pixel art specifically, FID < 10 against a reference set of shipped game sprites would be competitive. FID < 5 would be state-of-the-art for the niche.

Benchmark 2 — GenEval

Text-to-image alignment benchmark. Tests whether the generated image matches the prompt across categories: single object, two objects, counting, colors, position, attribute binding. Score 0-1.

Reference scores:

- PixelFlow: 0.64
- Stable Diffusion XL: 0.55
- DALL-E 3: 0.67
- Imagen 3: 0.66

Targeting: 0.60+ for general text-to-pixel-art alignment.

Benchmark 3 — DPG-Bench (Dense Prompt Generation Benchmark)

Tests text-to-image with complex multi-object prompts. Score 0-100.

Reference scores:

- PixelFlow: 77.93
- SD3: 84.08
- DALL-E 3: 83.50

Targeting: 75+ for pixel art workflows.

Benchmark 4 — PixelArena (NEW, 2025)

A pixel-precision visual intelligence benchmark using semantic segmentation tasks. Tests fine-grained generation rather than overall aesthetics. Established by NTU researchers.

Tested models: Gemini 3 Pro Image, GPT-4o, Emu series. Gemini 3 Pro Image showed "emergent image generation capabilities that generate semantic masks with high fidelity under zero-shot settings." Specific scores not yet broadly published.

This is the closest benchmark to "can the model produce pixel-precise output where each pixel is intentional." Worth targeting.

Benchmark 5 — Visual AI Index (VAI) / VCT2

The Visual Counter Turing Test. Measures how well AI-generated images can be detected as artificial. Higher score = harder to detect as AI = closer to human-quality. Not pixel-art-specific but used in mixed-media evaluation.

Section C — Practical Benchmark Targets For Your Project

Since pixel art doesn't have an industry-standard benchmark and PixelLab doesn't publish numbers, here's a practical benchmark suite to target:

Tier 1 — Match PixelLab's product capabilities:

- Generate from text prompt → pixel art at 64×64
- Generate from text prompt → pixel art at 128×128
- Generate from text prompt → pixel art at 256×256
- Generate from text prompt → pixel art at 400×400 (PixelLab's max)
- Reference-based generation (consistency across batches)
- 4-direction sprite rotation
- 8-direction sprite rotation
- Skeleton-based animation generation
- Walk/run/idle/attack animations from reference
- Wang tileset generation
- Sprite sheet PNG + JSON metadata export

Tier 2 — Quality benchmarks (real numbers to hit):

- FID < 10 against a reference set of 1000 shipped game sprites
- Zero "mixel" violations (all pixels are 1×1 at target resolution)
- Palette consistency: ≤ 16 unique colors per sprite
- Outline thickness: exactly 1px throughout silhouette
- Symmetry score > 90% for symmetric creatures
- Style consistency: < 5% palette drift across batch
- Generation time: < 5s per 64×64 sprite
- Animation frame consistency: ≤ 2px silhouette delta between consecutive idle frames

Tier 3 — Things PixelLab does NOT do (your differentiation):

- 3D model → pixel sprite bake (PixelLab is purely 2D generation)

- ASCII grid representation for AI-readable inspection
- Procedural creature library (slimes, wisps, blobs)
- Custom animation phase state machines
- Reference-comparison aesthetic scoring
- Open source / self-hostable
- Per-frame timing variation (most tools use uniform timing)
- Procedural shaders preserved through bake (gel, fire, etc)

Section D — Reference Datasets You Can Actually Use For FID

To compute FID against, you need a reference set. Real options:

Free / Open:

- Sprite-flow's training dataset: 128×128 RGBA pixel art characters
- Pokemon sprite datasets (multiple on Kaggle, ~800 sprites)
- Spriters Resource (community archive of game sprites, must scrape)
- OpenGameArt: thousands of CC-licensed pixel sprites
- Itch.io free asset packs (Kenney, etc.)

Academic:

- Multi-cell dataset (used in Pixelization SIGGRAPH 2022)
- Sprite generation datasets from Serpa & Rodrigues 2022
- Aliasing dataset (paired aliased/anti-aliased examples)

Commercial reference (study only, don't train on):

- PixelLab's gallery (visual reference for their style)
- Vampire Survivors sprite rips (community)
- Stardew Valley assets (extracted, fair use for benchmarking)

Section E — Honest Read On Competitive Positioning

PixelLab's REAL moat:

Not their model architecture (closed but probably standard diffusion + LoRA fine-tuning on pixel art datasets).

Their moat is:

1. Aseprite plugin distribution
2. Game dev community focus (Discord, tutorials)
3. MCP integration with AI coding tools
4. Vertical workflow (rotation, animation, tilesets) instead of just "generate one image"
5. First-mover brand recognition in pixel art AI

What you can realistically beat them on:

TECHNICAL:

- FID score (if you train on a more curated dataset)
- Output speed (if you optimize a smaller specialized model)
- Resolution flexibility (they cap at 400×400)
- True pixel-perfection (they have edge cases at higher resolutions)

ARCHITECTURAL:

- 3D-to-pixel pipeline (they're 2D-only, this is your unique angle)
- AI-native data interface (ASCII grids, structural inspection)
- Self-hostable / open source
- Procedural creature generation built in

WORKFLOW:

- Tighter integration with 3D modeling pipelines
- Reference comparison with structural metrics (not just style)
- Animation from procedural state machines (not skeletal mocap)

What you should NOT try to beat them on:

- Marketing reach

- Aseprite plugin (don't fight where they're entrenched)
- Generic "make me a pixel art character from a text prompt" (they're already there, model is already trained)

Section F — Recommended Benchmark Suite To Build Into Your Project

Build these as automated tests in your repo. Run on every commit.

Test 1 — Mixel Detection

For every output sprite, verify no pixels are larger than 1×1 at target resolution.

```
for each 2×2 block in output:
  if all 4 pixels same color AND surrounding pixels different:
    flag as potential mixel
fail if mixel ratio > 1%
```

Test 2 — Palette Discipline

Count unique colors. Must match declared palette size exactly.

```
unique = count_unique_rgba(output)
if unique > target_palette_size: fail
```

Test 3 — Outline Integrity

Walk silhouette boundary. Verify 1px thickness.

```
for every silhouette pixel:
  has_transparent_neighbor → must be outline color
  if has_outline_neighbor_in_2_directions → fail (2px thick)
```

Test 4 — FID Against Reference

Compute FID against curated 1000-sprite reference set.

- Initial target: FID < 30
- Production target: FID < 10
- SOTA target: FID < 5

Test 5 — Style Consistency

Generate same prompt 10 times. Compute pairwise palette overlap.

```
avg_overlap = mean(palette_jaccard(s_i, s_j))
must be > 0.85
```

Test 6 — Animation Stability

For idle animation, measure silhouette delta between consecutive frames.

```
delta = pixels_changed(frame_i, frame_i+1)
for idle: max delta should be ≤ 4px (sub-pixel animation rule)
```

Test 7 — Generation Speed

Time end-to-end generation.

- Target: < 5s for 64×64
- Target: < 15s for 128×128
- Target: < 30s for 256×256

Test 8 — Symmetry (for symmetric creatures)

Reflect output across vertical axis. Compute pixel match %.

- Target: > 90% for slimes, generic creatures
- Target: > 80% for biased creatures (asymmetric features)

Section G — What A “Win” Looks Like

A demo that beats PixelLab in a head-to-head:

1. Same input prompt: “fantasy slime creature, dark fantasy, gold and crimson palette”
2. Your tool produces:
    - Equal or better visual quality (FID-comparable)
    - 4-directional sprite sheet
    - 6-frame attack animation with proper per-frame timing
    - Procedural variations (10 slime variants from one prompt)

    - 3D model as bonus output (their tool can't do this)
3. PixelLab produces:
    - Same prompt result via PixFlux
    - 4-directional via their rotation feature
    - Animation via their skeleton system
    - Single output, no procedural variants
    - No 3D model
4. Both outputs run through the same FID evaluation against the same reference set. Yours scores within 20% of theirs.
5. Your tool runs locally / self-hosted; theirs requires their cloud and a subscription.

That's a defensible competitive position — not "we beat their model" (probably won't, they have years of training data), but "we cover the 3D-to-pixel niche they can't, with comparable 2D quality, and we're open source."

Section H — Realistic Expectations

Three honest things to know going in:

1. PixelLab has years of accumulated training data and product polish. A solo dev catching up on raw text-to-pixel-art quality is unlikely. Don't try. Win on architecture and workflow instead.
2. The "AI-native data pipeline" angle (3D source → grid data → render) is genuinely differentiated. No commercial competitor does this. This is where you have an actual moat.
3. Open source matters. PixelLab is closed and subscription-based. Game devs increasingly want self-hostable tools. Building this open source with a permissive license is itself a competitive advantage, regardless of raw model quality.

The benchmark you're actually competing on isn't "FID score vs PixelLab." It's "what can a self-hostable, 3D-to-pixel, AI-native pipeline do that no commercial tool can." That's a winnable game.

FINAL NOTE

The AI working on this project is the user of this tool. It will use the pipeline to draw pixel art. So the pipeline must be designed around what the AI can actually do natively — read structured data, apply rules, encode patterns from references, emit data.

If you find yourself designing features that require the AI to have visual perception, you're designing the wrong tool. The whole point is to build a tool that doesn't require that.

Build the AI-native version. The translation to human-perceivable output happens at the boundary, not in the middle.